

Natural Computing

LMU Munich
summer term 2025

Thomas Gabor



Stellenangebote am Lehrstuhl für Mobile und Verteilte Systeme

Fullstack Entwickler (m/w/d)

- Fullstack- & App-Entwicklung mit Next.js, React, Expo
- Umsetzung kompletter Entwicklungszyklen
- Deployment auf Kubernetes
- Performanceoptimierung & Qualitätssicherung
- Kontinuierliches Upskilling in modernen Webtechnologien

Wochenstunden	9 - 19 h
Befristung	wiederkehrend 3 Monate (später ggf. längere Laufzeit)



Data Scientist (m/w/d)

- Datenanalyse im Gesundheitskontext
- Entwicklung von Analysepipelines in Python
- Datenvisualisierung mit Matplotlib & Co.
- Toolentwicklung & Testing
- Unterstützung bei Auswertung & Dokumentation

Wochenstunden	9 - 19 h
Befristung	wiederkehrend 3 Monate (später ggf. längere Laufzeit)



Announcement:

Python Crash Course

on Thursday, May 8

target audience:

students who are familiar with another programming language
and want to catch up on Python specifics

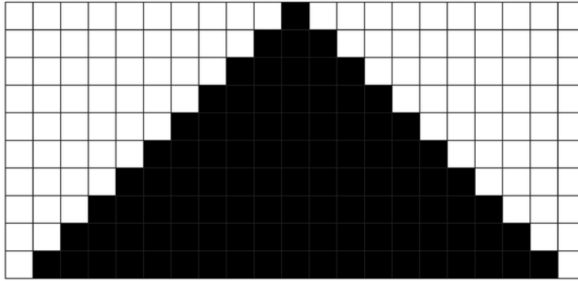
self-study material on basics available on Moodle! (look for “Exercise 0”)

Local information
is a constraint on
what we can do
efficiently

Is local information sufficient
for meaningful computation?

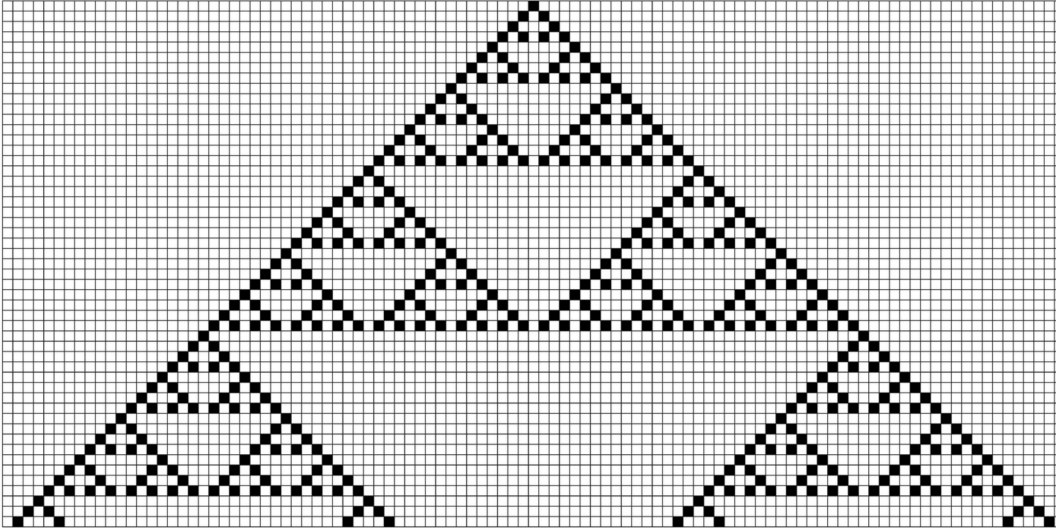
When I can use
only local information,
I might be more ---
- physical - robust

source:
wolframscience.com/nks

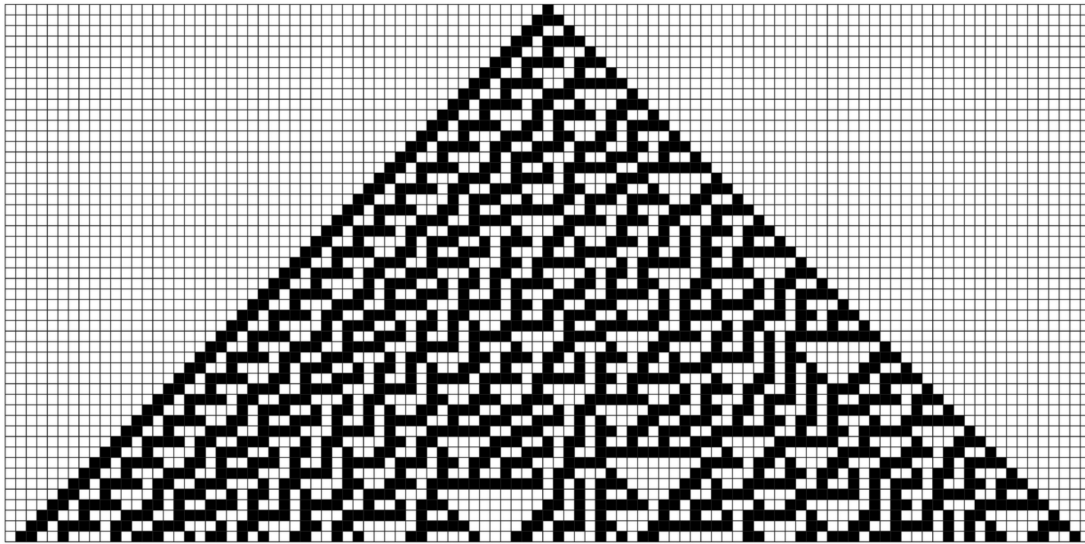


gives one of the possible combinations of colors for a cell and its immediate neighbors. The bottom row then specifies what color the center cell should be on the next step in each of these cases. In the numbering scheme described in Chapter 3, this is cellular automaton rule 254.

A representation of the rule for the cellular automaton shown above. The top row in each box



A cellular automaton that produces an intricate nested pattern. The rule in this case is that a cell should be black whenever one or the other, but not both, of its neighbors were black on the step before. Even though the rule is very simple, the picture shows that the overall pattern obtained over the course of 50 steps starting from a single black cell is not so simple. The particular rule used here can be described by the formula $a'_i = \text{Mod}[a_{i-1} + a_{i+1}, 2]$. In the numbering scheme of Chapter 3, it is cellular automaton rule 90.



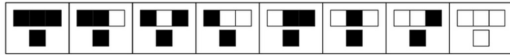
A cellular automaton with a simple rule that generates a pattern which seems in many respects random. The rule used is of the same type as in the previous examples, and the cellular automaton is again started from a single black cell. But now the pattern that is obtained is highly complex, and shows almost no overall regularity. This picture is our first example of the fundamental phenomenon that even with simple underlying rules and simple initial conditions, it is possible to produce behavior of great complexity. In the numbering scheme of Chapter 3, the cellular automaton shown here is rule 30.

On Names for Rules



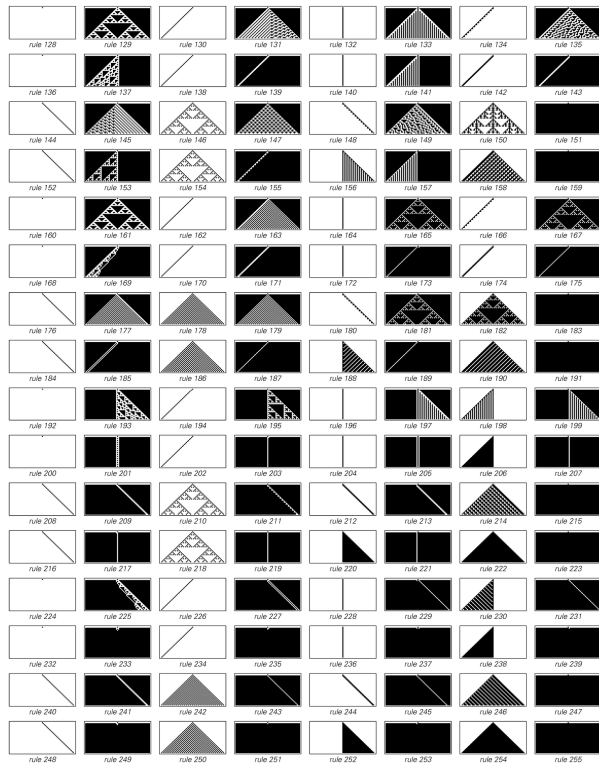
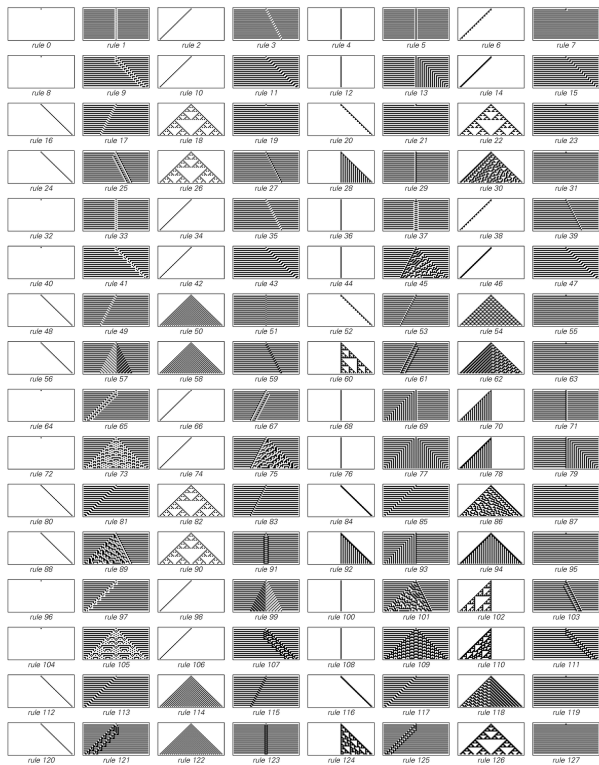
A cellular automaton that produces an intricate nested pattern. The rule in this case is that a cell should be black whenever one or the other, but not both, of its neighbors were black on the step before. Even though the rule is very simple, the picture

shows that the overall pattern obtained over the course of 50 steps starting from a single black cell is not so simple. The particular rule used here can be described by the formula $a_i' = \text{Mod}[a_{i-1} + a_{i+1}, 2]$. In the numbering scheme of Chapter 3, it is cellular automaton rule 90.



A representation of the rule for the cellular automaton shown above. The top row in each box

gives one of the possible combinations of colors for a cell and its immediate neighbors. The bottom row then specifies what color the center cell should be on the next step in each of these cases. In the numbering scheme described in Chapter 3, this is cellular automaton rule 254.



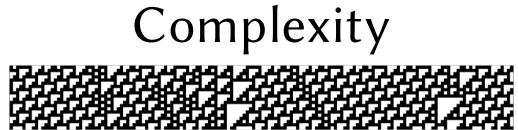
Classes of Cellular Automata



Uniformity



Repetition



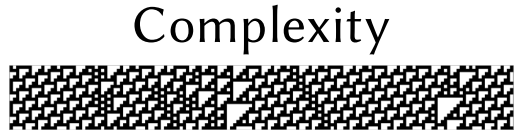
Classes of Cellular Automata



Uniformity



Repetition





source: commons.wikimedia.org/wiki/File:Textile_cone.JPG

Outlook: Neural Cellular Automata

Outlook: Neural Cellular Automata



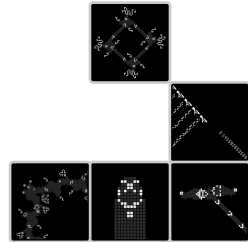
Let's try

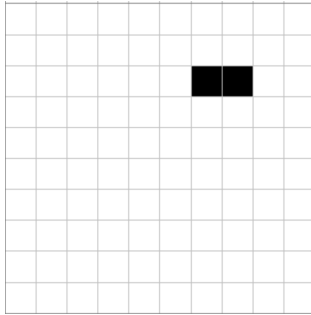
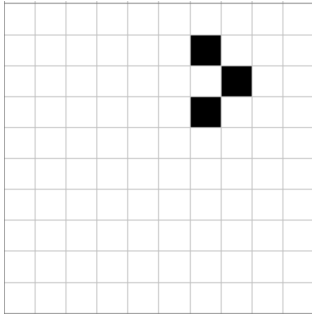
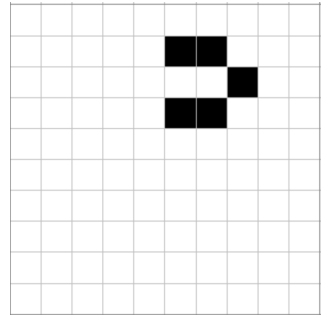
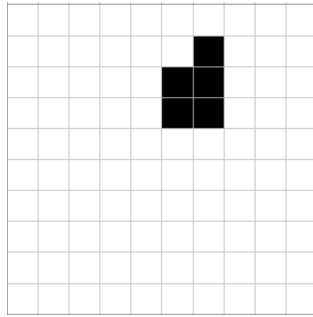
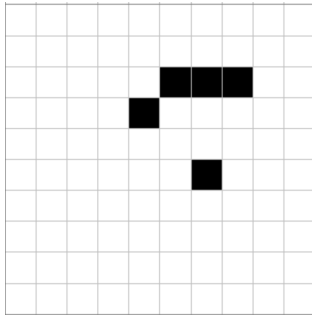
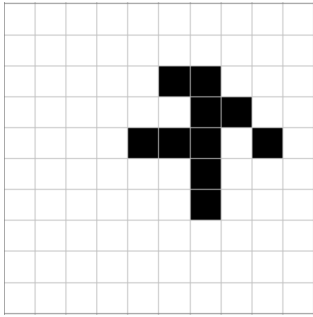
<https://distill.pub/2020/growing-ca/>

The Game of Life — Again

Johnston, Greene.
Conway's Game of Life — Mathematics and Construction.
Online, 2022.

conwaylife.com/book/





Definition 3 (Conway's game of life (standard)). Let $G = (V, E)$ be a graph with vertices V and (undirected) edges $E \subseteq V \times V$ with a fixed degree of 8 for all nodes. We define $surroundings : V \rightarrow \wp(V)$ via

$$surroundings(v) = \{w \mid (v, w) \in E\}$$

so that $|surroundings(v)| = 8$ and $v \notin surroundings(v)$ for all $v \in V$. A state $x \in \mathcal{X}$ is a mapping of vertices to the labels $\{dead, alive\}$, i.e., the state space $\mathcal{X}$ is given via $\mathcal{X} = (V \rightarrow \{dead, alive\})$. Let x_t be a state that exists at time step $t \in \mathbb{N}$. We define

$$|v|_{x_t} = |\{w \mid w \in surroundings(v) \wedge x_t(w) = alive\}|.$$

In the game of life, the evolution of a state x_t to its subsequent state x_{t+1} is given deterministically via

$$x_{t+1}(v) = \begin{cases} dead & \text{if } |v|_{x_t} \leq 1, \\ x_t(v) & \text{if } |v|_{x_t} = 2, \\ alive & \text{if } |v|_{x_t} = 3, \\ dead & \text{if } |v|_{x_t} \geq 4, \end{cases}$$

for all $v \in V$. A tuple (G, x_0) is called an instance of the game of life for initial state $x_0 \in \mathcal{X}$.

Let's try

conwaylife.com

Still Lifes

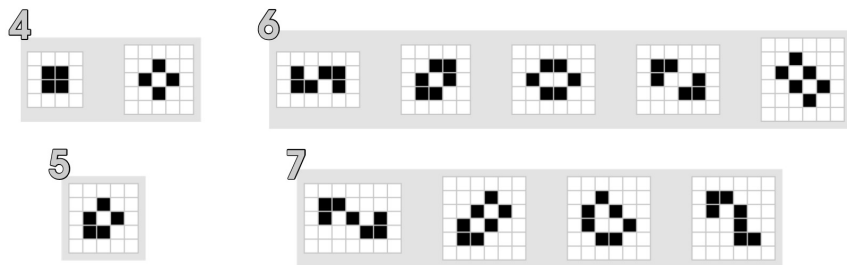


Figure 2.1: All still lifes with 7 or fewer live cells, arranged by their cell count. From left to right: (4 cells) block, tub, (5 cells) boat, (6 cells) snake, ship, beehive, aircraft carrier, barge, (7 cells) long snake, long boat, loaf, and eater 1 (which is sometimes called **fishhook**).

source: conwaylife.com/book/

Oscillators

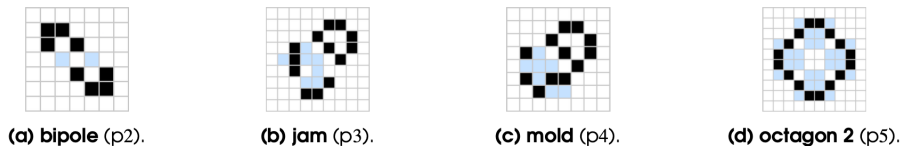


Figure 3.1: Some more small naturally occurring (but rare) oscillators that can be found via computer-assisted soup searches. These oscillators were found by (a) early Lifenthusiasts at M.I.T. in 1970, (b,c) Achim Flammenkamp in 1988,² and (d) Sol Goodman and Arthur Taber in 1971.

source: conwaylife.com/book/

Space Ships

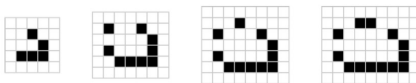
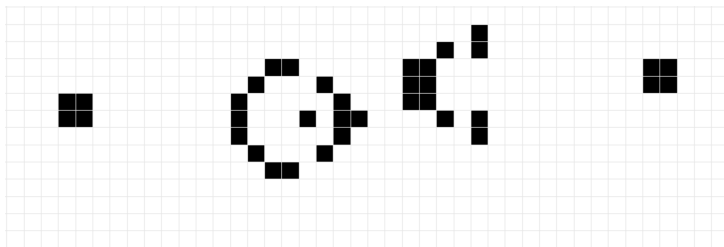


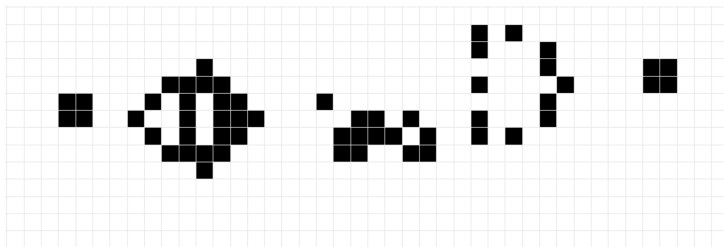
Figure 4.1: The four basic spaceships in Conway's Game of Life. From left to right, these are the glider (which moves diagonally at a speed of $c/4$) and the light/middle/heavyweight spaceships (which each move orthogonally¹ at a speed of $c/2$).

source: conwaylife.com/book/

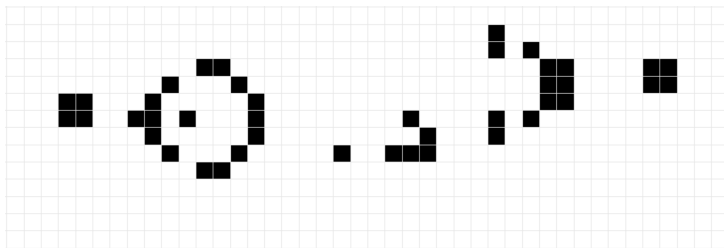
Guns



t = 1



t = 14



t = 15

[try online](#)

source: conwaylife.com

Space Ship Maneuvering

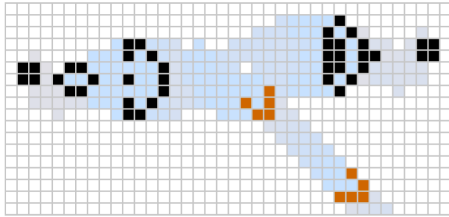


Figure 6.2: A Gosper glider gun producing gliders at a spacing of 30 generations.

source: conwaylife.com/book/

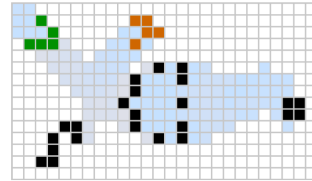


Figure 6.3: A buckaroo can reflect a glider by 90 degrees, from the position marked in green to the one in orange 30 generations later.

More Space Ship Maneuvering

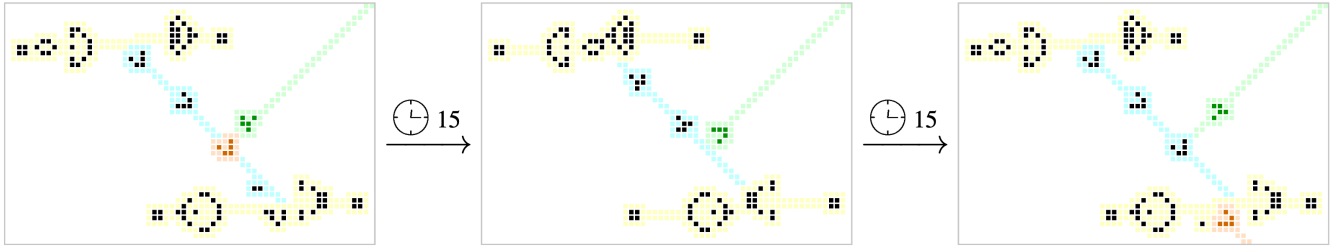


Figure 6.7: Two Gosper glider guns can be placed near each other so as to create a finite stream of gliders (highlighted here in aqua) between them. Here, we bounce a single glider (in green) off of one of those gliders (in orange) so that it is destroyed and thus released by the inline inverter to the southeast.

source: conwaylife.com/book/

conwaylife.com/book/periodic_circuitry

- No initial pattern with simply proven unlimited growth
- Initial patterns with apparently unlimited growth
- Simple initial patterns with behavior for a long period of time

Gardner, 1970.

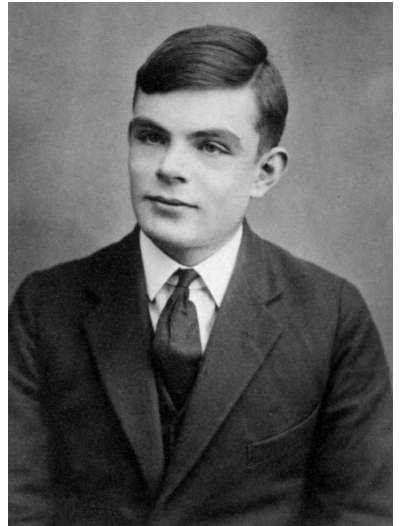
Why is this interesting?

Turing-Completeness

Alan Turing

1912 – 1954

- invented the field of (theoretical) computer science and artificial intelligence
- influential in breaking Enigma and thus shortening WWII



source: Wikipedia

The Turing Machine

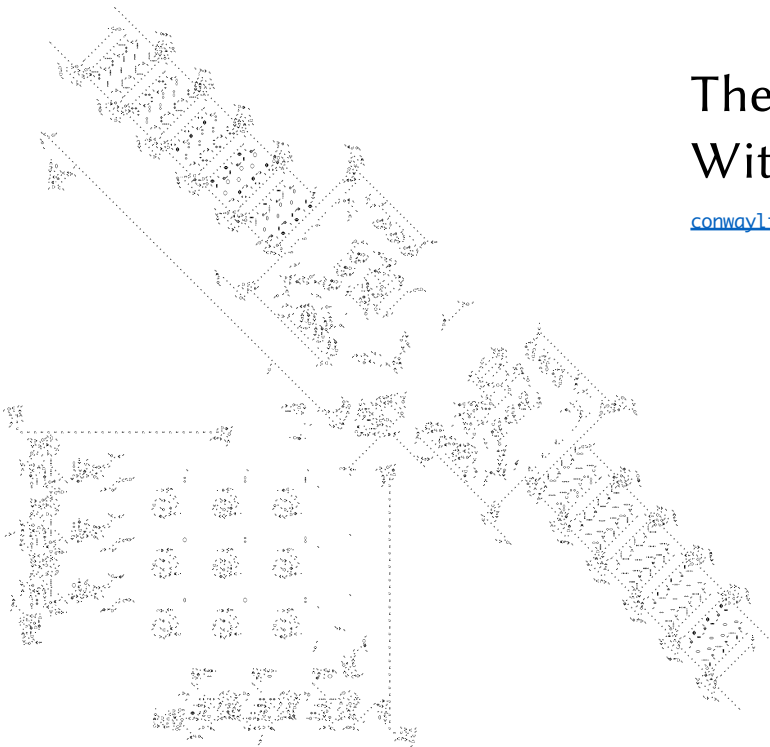
The Turing Machine

Definition 4 (Turing machine [1]). A Turing machine is a tuple $(Q, \Sigma, \Gamma, \sqcup, q_S, q_F, \delta)$ so that

- Q is a finite set of states,
- Σ is a finite input/output alphabet (e.g., $\Sigma = \{0, 1\}$),
- Γ is a finite tape alphabet with $\Sigma \subset \Gamma$,
- $\sqcup \in \Gamma$ is a blank symbol with $\sqcup \in \Sigma$,
- $q_S \in Q$ is a start state,
- $q_F \in Q$ is a stop state,
- and $\delta : Q \setminus \{q_F\} \times \Gamma \rightarrow Q \times \Gamma \times \{L, R, N\}$ is a partially defined transition function.

The Turing Machine Within the Game of Life

conwaylife.com/wiki/Turing_machine



But... why?

Can cellular automata
explain the universe?

Example 1 (Cellular Automaton for 1D Particle Collision, inspired by [1]). Consider the following variation of a 1D cellular automaton (cf. Definitions 1 and 2 on the definition sheet): Let the state space $\mathcal{X} = (V \rightarrow \mathbb{Z} \times \mathbb{Z})$. Let

$$f((v_l, p_l), (v_c, p_c), (v_r, p_r)) = (v_l^+ + v_r^- + \lfloor \frac{(p_l - p_c)^+}{2} \rfloor - \lfloor \frac{(p_r - p_c)^+}{2} \rfloor, \\ p_c - |v_c| + v_l^+ - v_r^- + \lfloor \frac{(p_l - p_c)^+}{2} \rfloor + \lfloor \frac{(p_r - p_c)^+}{2} \rfloor - \lfloor \frac{(p_c - p_l)^+}{2} \rfloor - \lfloor \frac{(p_c - p_r)^+}{2} \rfloor)$$

$$\text{where } x^+ = \begin{cases} x & \text{if } x > 0, \\ 0 & \text{otherwise,} \end{cases} \text{ and } x^- = \begin{cases} x & \text{if } x < 0, \\ 0 & \text{otherwise,} \end{cases} \text{ for any } x \in \mathbb{Z}.$$

Note: $\lfloor x \rfloor$ for $x \in \mathbb{R}$ is x rounded down to an integer as implemented in a typical `floor` function (see Python, e.g.).
 From the start configuration of this 1d cellular automaton with 5 cells below, compute the next 3 time steps.

(0, 0)	(+1, +1)	(0, 0)	(−1, +1)	(0, 0)

```

1 from math import floor
2
3 def eplus(x):
4     return x if x > 0 else 0
5
6 def eminus(x):
7     return x if x < 0 else 0
8
9 def f(left,center,right):
10     vl,pl = left
11     vc,pc = center
12     vr,pr = right
13     pressure_from_left = floor(eplus(pl-pc)/2)
14     pressure_from_right = floor(eplus(pr-pc)/2)
15     pressure_towards_left = floor(eplus(pc-pl)/2)
16     pressure_towards_right = floor(eplus(pc-pr)/2)
17     vnew = eplus(vl) + eminus(vr) + pressure_from_left - pressure_from_right
18     pnew = pc - abs(vc) + eplus(vl) - eminus(vr) + pressure_from_left + pressure_from_right - pressure_towards_left - pressure_towards_right
19     return vnew, pnew
20
21 def apply(vector, f):
22     new_vector = []
23     for e,element in enumerate(vector):
24         new_element = f(vector[e-1], element, vector[(e+1)%len(vector)])
25         new_vector.append(new_element)
26     return new_vector
27
28 rounds = range(4)
29 vector = [(0,0), (+1,+1), (0,0), (-1,+1), (0,0)]
30 for _ in rounds:
31     print(vector)
32     vector = apply(vector,f)

```

Example 1 (Cellular Automaton for 1D Particle Collision, inspired by [1]). Consider the following variation of a 1D cellular automaton (cf. Definitions 1 and 2 on the definition sheet): Let the state space $\mathcal{X} = (V \rightarrow \mathbb{Z} \times \mathbb{Z})$. Let

$$f((v_l, p_l), (v_c, p_c), (v_r, p_r)) = (v_l^+ + v_r^- + \lfloor \frac{(p_l - p_c)^+}{2} \rfloor - \lfloor \frac{(p_r - p_c)^+}{2} \rfloor, \\ p_c - |v_c| + v_l^+ - v_r^- + \lfloor \frac{(p_l - p_c)^+}{2} \rfloor + \lfloor \frac{(p_r - p_c)^+}{2} \rfloor - \lfloor \frac{(p_c - p_l)^+}{2} \rfloor - \lfloor \frac{(p_c - p_r)^+}{2} \rfloor)$$

$$\text{where } x^+ = \begin{cases} x & \text{if } x > 0, \\ 0 & \text{otherwise,} \end{cases} \text{ and } x^- = \begin{cases} x & \text{if } x < 0, \\ 0 & \text{otherwise,} \end{cases} \text{ for any } x \in \mathbb{Z}.$$

Note: $\lfloor x \rfloor$ for $x \in \mathbb{R}$ is x rounded down to an integer as implemented in a typical `floor` function (see Python, e.g.).
 From the start configuration of this 1d cellular automaton with 5 cells below, compute the next 3 time steps.

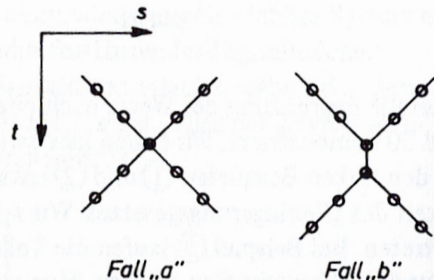
(0, 0)	(+1, +1)	(0, 0)	(−1, +1)	(0, 0)
(0, 0)	(0, 0)	(0, 2)	(0, 0)	(0, 0)
(0, 0)	(−1, +1)	(0, 0)	(+1, +1)	(0, 0)
(−1, +1)	(0, 0)	(0, 0)	(0, 0)	(+1, +1)

state space $\mathcal{X} = (V \rightarrow \mathbb{Z} \times \mathbb{Z})$

t_0	v								
	p				+1	+1			
t_1	v			-1		+1			
	p			+1			+1		
t_2	v			-1				+1	
	p		+1					+1	

state space $\mathcal{X} = (V \rightarrow \mathbb{Z} \times \mathbb{Z})$

t_0	v									
	p				+1	+1				
t_1	v				-1		+1			
	p			+1				+1		
t_2	v			-1					+1	
	p		+1							+1



Konrad Zuse

1910 – 1995

- built first programmable computer (Zuse Z3)
- designed first high-level programming language (Plankalkül)



source: Wikipedia

What about
non-local
information?